

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

11. Quan sát hệ thống — Observability & Distributed Tracing	2
11.1. Ba trụ cột của Observability	3
11.1.1. Vấn đề: “hệ thống lỗi nhưng không biết lỗi ở đâu”	3
11.1.2. Ba trụ cột	3
11.1.3. Observability vs Monitoring	4
11.1.4. Mô hình trưởng thành Observability (Observability Maturity Model)	5
11.2. Logging — từ console đến gom log tập trung	6
11.2.1. Vấn đề: logs phân tán, không có ngữ cảnh	6
11.2.2. Log có cấu trúc — từ văn bản thô sang JSON	6
11.2.3. Gom log tập trung (Centralized Log Aggregation)	7
11.2.4. Correlation IDs — kỹ thuật nền tảng	7
11.3. Chi tiết về Distributed Tracing	9
11.3.1. Vấn đề: request chậm — chậm ở đâu?	9
11.3.2. Distributed Tracing — tự động đo thời gian	9
11.3.3. Các khái niệm: Trace, Span, Context Propagation	10
11.3.4. Chiến lược lấy mẫu (Sampling)	10
11.3.5. Tracing trong Spring Boot 3.x	11
11.3.6. Kiến trúc triển khai Observability Pipeline	12
11.4. Metrics, SLI/SLO và chiến lược cảnh báo	12
11.4.1. Metrics — đo lường hành vi theo thời gian	12
11.4.2. Bốn loại Metrics cơ bản của Prometheus	13
11.4.3. Hai khung đo lường: RED và USE	13
11.4.4. SLI/SLO/SLA — từ metrics đến cam kết chất lượng	14
11.4.5. Chiến lược cảnh báo	15
11.5. Chiến lược xử lý lỗi — nhất quán giữa các service	16
11.5.1. Nguyên tắc thiết kế xử lý lỗi	16
11.6. Health Checks và Service Discovery	17
11.6.1. Health Checks — nền tảng cho tự động hóa	17
11.7. User Activity Tracking — Observability ở mức nghiệp vụ	18
11.8. Chaos Engineering — kiểm tra khả năng phục hồi (resilience) chủ động	19
11.8.1. Case study thực tế: Netflix Simian Army	20
11.9. Case study: KBM	21
11.9.1. Hiện trạng — đánh giá theo mô hình trưởng thành	21
11.9.2. Phân tích theo business context	21
11.10. Tổng kết	23
11.11. Đọc thêm	24
Tài liệu tham khảo	25

Quan sát hệ thống — Observability & Distributed Tracing

“Observability is not just monitoring, it’s the ability to ask new questions about your system’s behavior without deploying new code.”

— Charity Majors (tổng hợp từ observability community)

MỤC TIÊU HỌC TẬP

- Hiểu tại sao observability là **điều kiện tiên quyết** để vận hành microservices — không chỉ là thứ “có thì tốt”
- Phân biệt **observability** và **monitoring** và khi nào cần gì
- Nắm vững ba trụ cột: **Logs Metrics Traces** và cách chúng hỗ trợ nhau
- Thiết kế chiến lược **log tập trung** (centralized logging) với log có cấu trúc (structured logs) và correlation ID
- Hiểu **truy vết phân tán** (distributed tracing) và các khái niệm trace, span, context propagation
- Nắm framework đo lường: **SLI/SLO/SLA** và chiến lược cảnh báo (alerting)
- Xây dựng **chiến lược xử lý lỗi** nhất quán giữa các service
- Phân tích các khoảng trống observability trong KBM và đề xuất lộ trình chuyển đổi

Lưu ý

Ghi chú Chương này tập trung vào nguyên tắc và thiết kế observability độc lập với hạ tầng triển khai cụ thể. Sau khi đọc Chương 12 (Triển khai & DevOps), người đọc sẽ thấy rõ hơn cách các công cụ observability được tích hợp vào CI/CD pipeline và môi trường chạy container.

Để vận hành hệ thống phân tán, các nhóm phát triển cần chuyển từ tư duy giám sát thụ động (Monitoring: hệ thống có đang hoạt động bình thường không?) sang khả năng quan sát chủ động (Observability: tại sao hệ thống chạy chậm?).

11.1. Ba trụ cột của Observability

Để đảm bảo hệ thống LMS hoạt động trơn tru, chúng ta cần xây dựng cơ chế giám sát dựa trên ba trụ cột cốt lõi, giúp định vị chính xác mọi sự cố.

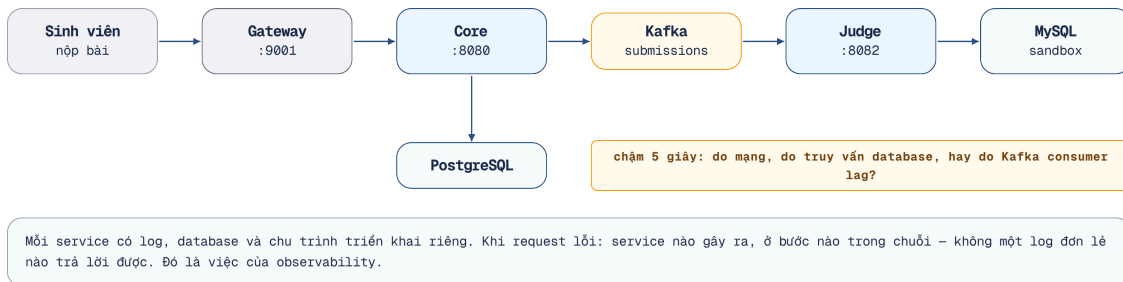
11.1.1. Vấn đề: “hệ thống lỗi nhưng không biết lỗi ở đâu”

Trong monolith, khi xảy ra lỗi: chỉ cần mở log file duy nhất, tìm ngoại lệ (exception) và đọc stack trace là có thể định vị được vấn đề. Toàn bộ request chạy trong cùng tiến trình, cùng log file, cùng database.

Trong kiến trúc microservices, một yêu cầu từ người dùng có thể đi qua nhiều dịch vụ, mỗi dịch vụ có nhật ký hệ thống, cơ sở dữ liệu và chu trình triển khai độc lập. Khi một yêu cầu gặp lỗi:

- Service nào gây lỗi? Gateway? Core? Judge? Auth?
- Lỗi xảy ra ở bước nào trong chuỗi gọi?
- Độ trễ 5 giây — do mạng, do truy vấn database, hay do Kafka consumer lag?

Hình 11.1 cho thấy một request đi qua hơn năm service — lỗi có thể nằm ở bất kỳ chặng nào.

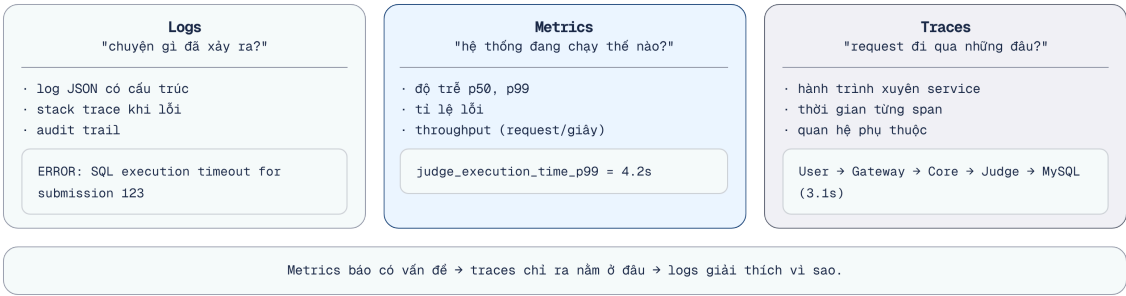


Hình 11.1: Luồng request qua 5+ services — lỗi xảy ra ở đâu?

Newman trong [4a, Ch.8] gọi đây là một thách thức lớn khi vận hành microservices: **sự phức tạp không nằm ở code, mà nằm ở tương tác giữa các services**. Trong monolith, lỗi thường có stack trace rõ ràng — một file, một method. Trong microservices, lỗi có thể xuất hiện ở dịch vụ B do dịch vụ A gửi dữ liệu sai, hoặc do message Kafka bị trễ — lúc này, không có stack trace nào ghi lại được nguyên nhân thật sự.

11.1.2. Ba trụ cột

Richardson trong [2a, Ch.11] và Newman trong [4a, Ch.8] đều mô tả observability qua **ba trụ cột**:



Hình 11.2: Ba trụ cột của Observability — Logs, Metrics, Traces

Hình 11.2 minh họa ba trụ cột ở mức tổng quan: mỗi trụ cột phục vụ một loại câu hỏi khác nhau khi debug hệ thống phân tán. Bảng dưới đây mô tả chi tiết vai trò, ví dụ cụ thể trong KBM, và các công cụ phổ biến cho từng trụ cột — giúp lập trình viên hình dung cách áp dụng ngay vào hệ thống thực tế.

Bảng 11.1: Vai trò của ba trụ cột Observability

Trụ cột	Trả lời câu hỏi	Ví dụ trong KBM	Công cụ phổ biến
Logs	“Chuyện gì đã xảy ra?”	ERROR: SQL execution timeout for submission 123	ELK Stack, Loki, Fluentd
Metrics	“Hệ thống đang hoạt động thế nào?”	judge_execution_time_p99 = 4.2s	Prometheus, Grafana, Micrometer
Traces	“Request đi qua những đâu?”	User -> Gateway -> Core -> Judge -> MySQL (total: 3.1s)	Jaeger, Grafana Tempo, Zipkin (Dùng chuẩn OpenTelemetry OTLP)

M.E.L.T và Events

Trong công nghiệp, nhiều nhà cung cấp (New Relic, Datadog) dùng thuật ngữ **M.E.L.T**, nâng chữ **E** — **Events** lên thành trụ cột thứ tư. Tuy nhiên với hệ phân tán, Events mang nghĩa nghiệp vụ nên thường đi riêng qua Message Broker (Kafka) thay vì dùng chung đường ống giám sát kỹ thuật.

Ba trụ cột **kết hợp với nhau**: metrics cho biết *hệ thống đang có vấn đề* (thời gian phản hồi tăng), traces chỉ ra *vấn đề nằm ở đâu* (Dịch vụ Judge chiếm 90% thời gian xử lý), logs giải thích nguyên nhân *tại sao* (truy vấn SQL bị quá hạn do thiếu chỉ mục). Thiếu bất kỳ trụ cột nào, hai thành phần còn lại cũng mất đáng kể giá trị.

11.1.3. Observability vs Monitoring

Bảng 11.2 phân biệt hai khái niệm thường bị dùng lẫn này.

Bảng 11.2: Observability vs Monitoring

	Monitoring	Observability
Mục đích	Phát hiện vấn đề đã biết	Khám phá vấn đề chưa biết
Cách tiếp cận	Đặt cảnh báo cho metrics cụ thể	Khám phá dữ liệu để hiểu hành vi hệ thống

	Monitoring	Observability
Ví dụ	“CPU > 80% → cảnh báo”	“Tại sao độ trễ tăng gấp 3 lúc 2 giờ chiều mỗi thứ Ba?”
Dữ liệu	Dashboard định sẵn	Logs + Metrics + Traces (kết hợp tùy biến theo nhu cầu)
Khi nào đủ	Hệ thống đơn giản, các kiểu lỗi (failure modes) biết trước	Hệ thống phân tán, các kiểu lỗi không dự đoán được

Monitoring trả lời: “*Có vấn đề không?*” Observability trả lời: “*Tại sao có vấn đề này, và vấn đề này khác gì so với lần trước?*” Với monolith, monitoring thường đủ. Với microservices — nơi các kiểu lỗi phức tạp và không dự đoán được — observability là yêu cầu bắt buộc.

Observability ≠ Monitoring

“Monitoring tells you *that* something is wrong. Observability lets you ask *why*. Monitoring is a subset of observability — necessary but not sufficient.”

— Tổng hợp từ Charity Majors và cộng đồng observability

Ảnh dụ LMS: camera giám sát vs hệ thống quản trị

- **Monitoring:** Giống như bác bảo vệ nhìn vào màn hình **camera giám sát** và thấy sinh viên đang ùn tắc trước thang máy nhà A. Bác chỉ biết: *Hệ thống đang có vấn đề.*
- **Observability:** Bác bảo vệ có hệ thống liên kết camera với lịch học, thẻ quét sinh viên. Tra cứu nhanh, bác phát hiện ra: *Có 2 lớp học bù đột xuất được xếp trùng giờ ở cùng tầng 9.* Bác biết chính xác: *Tại sao có vấn đề và nguyên nhân gốc rễ là gì.*

11.1.4. Mô hình trưởng thành Observability (Observability Maturity Model)

Không phải hệ thống nào cũng cần đầy đủ bộ công cụ observability ngay từ đầu. Mô hình trưởng thành giúp xác định *đang ở đâu* và *cần đến đâu*.

Bảng 11.3: Mô hình trưởng thành Observability

Cấp độ	Mô tả	Đặc điểm	Phù hợp với
1 — Reactive	Debug khi người dùng báo lỗi	Console logs, không có cảnh báo tự động, điều tra thủ công	Dự án cá nhân, bản mẫu thử nghiệm
2 — Basic monitoring	Dashboards cơ bản, cảnh báo thô	Health checks, CPU/memory metrics, log tập trung	Nhóm nhỏ, hệ thống ít dịch vụ
3 — Proactive	Phát hiện vấn đề trước khi người dùng gặp phải	Log có cấu trúc, correlation ID, metrics tùy biến, cảnh báo	Nhóm quy mô vừa, nhiều dịch vụ
4 — Observability	Hỏi câu hỏi mới không cần code mới	Distributed tracing, SLI/SLO, high-cardinality data	Nhóm quy mô lớn, hệ thống phức tạp
5 — Predictive	Dự đoán vấn đề trước khi xảy ra	Phát hiện bất thường dựa trên ML, hoạch định năng lực	Doanh nghiệp lớn, hệ thống trọng yếu

Nguồn: Tổng hợp từ Richardson [2a, Ch.11], Newman [4a, Ch.8], và Charity Majors, Liz Fong-Jones, George Miranda, Observability Engineering (O'Reilly, 2022). Các cấp độ trưởng thành phản ánh kiến thức cộng đồng observability — không theo một framework độc quyền nào.

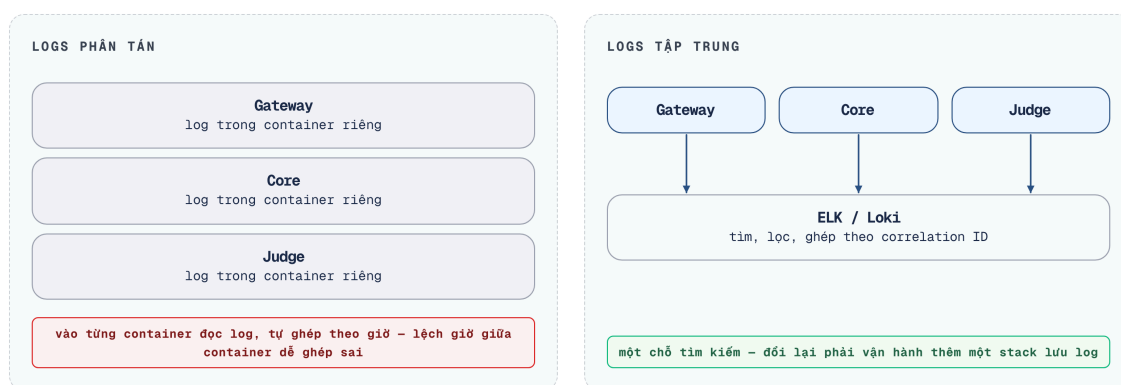
Với LMS — hệ thống giáo dục quy mô trung bình — mục tiêu hợp lý là **Cấp 3** (Proactive), với khả năng mở rộng lên Cấp 4 khi cần.

11.2. Logging — từ console đến gom log tập trung

Trong kiến trúc microservices, một request từ người dùng đi qua nhiều dịch vụ. Nếu logs nằm rải rác trên nhiều máy chủ khác nhau, việc truy vết lỗi trở nên tốn kém và dễ bỏ sót.

11.2.1. Vấn đề: logs phân tán, không có ngữ cảnh

Khi hệ thống có 7+ services chạy đồng thời, mỗi service ghi log vào container riêng. Để truy vết một yêu cầu thất bại, lập trình viên phải kết nối (SSH) vào từng container, đọc nhật ký và chấp vá thời gian — rất khó với các request đồng thời và lệch thời gian (clock skew) giữa containers.



Hình 11.3: Logs phân tán (gỡ lỗi thủ công) vs Logs tập trung — tìm kiếm tại một nơi

11.2.2. Log có cấu trúc — từ văn bản thô sang JSON

Bước đầu tiên: chuyển từ **log dạng văn bản** (text logs) sang **log có cấu trúc** (JSON). Log dạng văn bản dễ đọc cho người nhưng rất khó phân tích tự động.

Listing 11.1: Ví dụ log có cấu trúc (JSON)

```
// SAI: Text log — khó parse, khó filter, khó aggregate
2026-01-15 10:03:02 ERROR [kbm-core] - SQL execution failed for submission 8a3f
user 12ab, Timeout after 30s

// ĐÚNG: Structured log (JSON) — dễ query, filter, aggregate
{"timestamp":"2026-01-15T10:03:02Z","level":"ERROR","service":"kbm-core",
 "message":"SQL execution failed","submissionId":"8a3f","userId":"12ab",
 "duration_ms":30000,"error":"QueryTimeoutException","traceId":"abc123"}
```

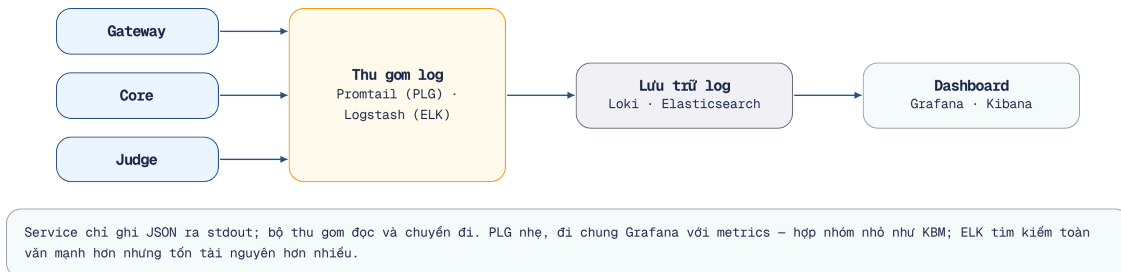
Với log có cấu trúc, hệ thống có thể truy vấn: “Tìm tất cả các lỗi trong 24h qua có **error = QueryTimeoutException**” hoặc “Thời gian xử lý trung bình của các lượt nộp bài theo giờ” — điều không thể làm với log dạng văn bản.

Listing 11.2: Khai báo Logback xuất JSON chứa Trace/Span ID (*logback-spring.xml*)

```
<configuration>
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <!-- Tự động trích xuất traceId, spanId từ MDC (nếu dùng Micrometer
Tracing) -->
      <includeMdcKeyName>traceId</includeMdcKeyName>
      <includeMdcKeyName>spanId</includeMdcKeyName>
    </encoder>
  </appender>
  <root level="INFO"><appender-ref ref="CONSOLE" /></root>
</configuration>
```

11.2.3. Gom log tập trung (Centralized Log Aggregation)

Khi log đã có cấu trúc, bước tiếp theo là **gom log tập trung** — gom logs từ tất cả services về một nơi. Newman trong [4a, Ch.8] nhấn mạnh: *log tập trung là yêu cầu bắt buộc* cho microservices, không phải tùy chọn.

**Hình 11.4:** Pipeline gom logs tập trung — từ service đến nơi lưu trữ

Để triển khai một pipeline thu thập log tập trung hiệu quả như minh họa ở trên, các tổ chức thường phải đánh đổi giữa tính năng tìm kiếm toàn văn bản (full-text search) chuyên sâu và mức độ tiêu thụ tài nguyên phần cứng. Trong bối cảnh hệ sinh thái cloud-native, có rất nhiều công cụ hỗ trợ nhưng nổi bật nhất vẫn là hai stack công nghệ tiêu chuẩn dưới đây:

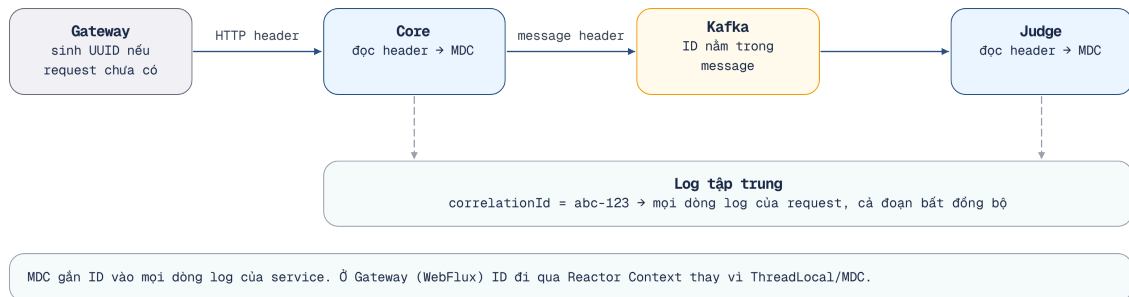
Bảng 11.4: So sánh hai stack ELK và PLG

Stack	Thành phần	Ưu điểm	Nhược điểm	Phù hợp
ELK	Elasticsearch + Logstash + Kibana	Trường thành, tìm kiếm toàn văn mạnh	Tốn tài nguyên (tối thiểu ~4GB RAM)	Nhóm quy mô lớn, nhiều logs
PLG	Promtail (nay là Grafana Alloy) + Loki + Grafana	Nhẹ, tích hợp metrics	Truy vấn kém linh hoạt hơn ELK	Nhóm nhỏ vừa, KBM

11.2.4. Correlation IDs — kỹ thuật nền tảng

Dù đã có hệ thống gom log tập trung xuất sắc, việc phân tích nguyên nhân gốc rễ (root cause analysis) vẫn sẽ rất khó khăn nếu các dòng log của cùng một request bị rời rạc. Để giải quyết vấn đề này, Newman trong [4a, Ch.8] mô tả **Correlation ID** là kỹ thuật nền tảng cho việc gỡ lỗi microservices. Cơ chế này yêu cầu hệ thống sinh ra một mã định danh duy nhất gắn vào request từ điểm vào (Gateway), sau đó truyền qua tất cả các services thông qua HTTP headers và cả Kafka message headers nhằm xâu chuỗi toàn bộ quá trình xử lý. Lưu ý: Correlation ID không đồng

ngĩa với Trace ID (W3C **traceparent**) — mỗi loại có vòng đời và chủ sở hữu riêng (xem §11.3 về Distributed Tracing).



Hình 11.5: Truyền Correlation ID xuyên suốt request, kể cả qua Kafka

🔔 Phiếu luân chuyển hồ sơ sinh viên

Hãy tưởng tượng một sinh viên nộp hồ sơ xin xét tốt nghiệp. Hồ sơ này được dán một mã vạch (Correlation ID) duy nhất. Bất kể hồ sơ đi qua Phòng Đào tạo (Core Service), Phòng Tài chính (Finance Service), hay Thư viện (Library Service), mọi phòng ban đều phải quét mã vạch này và ghi vào sổ nhật ký của họ. Nếu hồ sơ bị kẹt, Ban Giám hiệu chỉ cần tìm kiếm mã vạch đó là biết chính xác nó đang nằm ở phòng nào.

Cơ chế hoạt động:

1. **Gateway** sinh UUID cho mỗi request đi vào, gắn vào HTTP header **X-Correlation-Id**
2. **Downstream services** đọc header, ghi vào **MDC** (Mapped Diagnostic Context) — logging framework tự động gắn kèm vào mọi dòng log
3. **Kafka messages** truyền correlation ID qua message headers — nối luồng bất đồng bộ
4. **Khi gỡ lỗi**: tìm **correlationId = abc-123** trong hệ thống log tập trung → thấy toàn bộ vòng đời xử lý request

Ở Gateway, bước gắn correlation ID được viết thành một filter non-blocking (Listing 11.3):

Listing 11.3: Tiêm Correlation ID an toàn trong Spring Cloud Gateway (WebFlux, non-blocking)

```

@Component
public class CorrelationIdFilter implements GlobalFilter, Ordered {
    @Override
    public Mono<Void> filter(
        ServerWebExchange exchange, GatewayFilterChain chain) {
        String correlationId = exchange.getRequest().getHeaders()
            .getFirst("X-Correlation-Id");
        if (correlationId == null) {
            correlationId = UUID.randomUUID().toString();
        }

        // 1. Gắn vào HTTP Header để truyền cho downstream service
        ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
            .header("X-Correlation-Id", correlationId)
            .build();

        // 2. Lưu vào Reactor Context (KHÔNG DÙNG ThreadLocal/MDC ở đây)
    }
}

```

```
String finalId = correlationId;
return chain.filter(exchange.mutate().request(mutatedRequest).build())
    .contextWrite(Context.of("CORRELATION_ID", finalId));
}

@Override
public int getOrder() { return -1; }
}
```

⚠ **Mỗi request đều có một định danh xuyên suốt**

“Ensure that every request entering your system gets a unique identifier, and that this identifier is passed along to all downstream services. Without this, debugging distributed systems is virtually impossible.”

— Sam Newman, *Building Microservices* [4a, Ch.8]

11.3. Chi tiết về Distributed Tracing

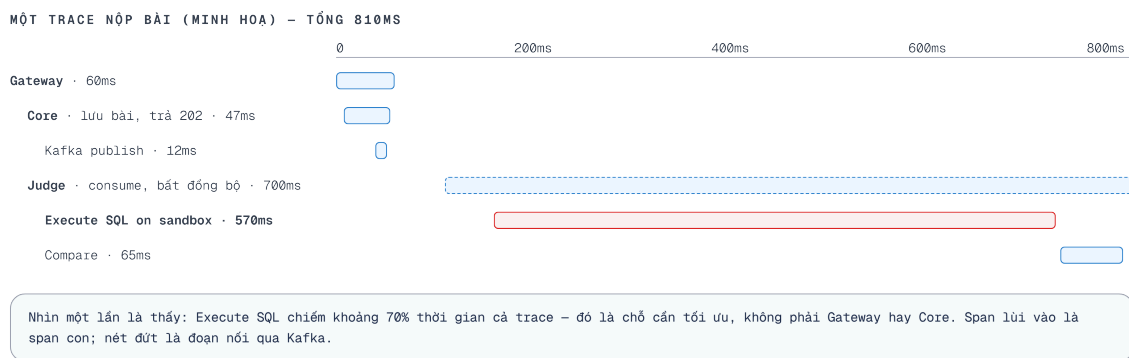
Khi sinh viên phàn nàn hệ thống nộp bài quá chậm, thật khó để biết độ trễ nằm ở đâu nếu không có cơ chế theo dõi dấu vết. Distributed Tracing giải quyết bài toán này bằng cách vẽ lại chính xác đường đi của mỗi yêu cầu.

11.3.1. Vấn đề: request chậm — chậm ở đâu?

Correlation IDs giúp nối logs lại — nhưng không cho biết **mỗi bước mất bao lâu**. Khi sinh viên nộp bài và thời gian phản hồi mất 8 giây (so với mức bình thường là 2 giây), lập trình viên biết yêu cầu đã đi qua Gateway → Core → Judge → MySQL, nhưng không thể xác định 6 giây dư thừa phát sinh ở bước nào nếu không dò timestamp trong log một cách thủ công.

11.3.2. Distributed Tracing — tự động đo thời gian

Distributed tracing ghi lại toàn bộ vòng đời của một request qua tất cả services, bao gồm thời gian mỗi bước (**span**):



Hình 11.6: Biểu đồ Gantt của một trace — thời gian của từng span

Từ trace này, rõ ràng: **Execute SQL on sandbox** mất 570ms — chiếm 70% tổng thời gian. Đây là nút thắt cổ chai (bottleneck) cần được tối ưu hóa. Không có tracing, lập trình viên buộc phải phỏng đoán, hoặc dò thủ công từng timestamp trong log.

11.3.3. Các khái niệm: Trace, Span, Context Propagation

🔗 Quy trình nhập học

- **Trace:** Là tổng thời gian từ lúc tân sinh viên bước vào trường đến lúc cầm thẻ sinh viên ra về (ví dụ: 120 phút).
- **Span:** Là thời gian sinh viên dừng lại ở từng trạm. Trạm “Đóng học phí” là một **span cha** (parent span, 30 phút).
- **Span con:** Bên trong trạm đóng học phí có 2 khâu nhỏ: “Xếp hàng chờ” (25 phút) và “Quẹt thẻ ngân hàng” (5 phút). Nhờ chia nhỏ thành span con (child span), chúng ta biết ngay “cổ chai” (bottleneck) không phải do hệ thống ngân hàng chậm, mà do thời gian chờ đợi xếp hàng quá lâu.

Ảnh dụ trên giúp hình dung trực quan quy trình tracing. Bảng dưới đây hệ thống hóa bốn khái niệm cốt lõi — Trace, Span, quan hệ cha-con, và Context propagation — cùng ví dụ cụ thể trong luồng nộp bài của KBM.

Bảng 11.5: Các khái niệm cơ bản trong Distributed Tracing

Khái niệm	Mô tả	Ví dụ
Trace	Toàn bộ vòng đời xử lý của một request	Nộp bài từ trình duyệt → trả điểm về
Span	Một bước trong trace (một đơn vị công việc)	“Core Service: save submission” (40ms)
Cha-con (parent-child)	Span có thể chứa các span con	“Judge Service” span chứa “Execute SQL” và “Compare”
Context propagation	Truyền trace context qua ranh giới service	HTTP header traceparent , Kafka message header

11.3.4. Chiến lược lấy mẫu (Sampling)

Trong môi trường thực tế (production), việc truy vết **100% requests** sẽ tiêu tốn rất nhiều dung lượng lưu trữ và năng lực xử lý. Do đó, cần có chiến lược lấy mẫu (sampling strategy):

Bảng 11.6: Các chiến lược lấy mẫu cho tracing

Chiến lược	Mô tả	Khi nào dùng
Head-based	Quyết định lấy mẫu tại điểm vào (Gateway)	Đơn giản, dễ triển khai
Tail-based	Quyết định sau khi trace hoàn thành (giữ traces lỗi, bỏ traces thành công)	Giữ 100% trace lỗi, giảm dung lượng lưu trữ
Rate-based	Lấy mẫu N% request	Production lưu lượng cao (1-10%)
Adaptive	Tự động điều chỉnh tỷ lệ lấy mẫu theo lưu lượng	Doanh nghiệp lớn, tải biến động

Với LMS: chế độ thi (100+ người dùng đồng thời) nên dùng rate-based 10-20%; ngoài kỳ thi, KBM (tài thấp, chạy trên một máy chủ) có thể dùng 100% để thuận tiện debug — nhưng đây là đặc thù hệ thống nhỏ; với production có lưu lượng lớn, cần chọn chiến lược sampling phù hợp (probabilistic, tail-based) dựa trên tải thực tế và chi phí lưu trữ.

11.3.5. Tracing trong Spring Boot 3.x

Spring Boot 3.x tích hợp sẵn **Micrometer Tracing** — lớp trừu tượng (abstraction layer) cho distributed tracing. **OpenTelemetry (OTel)** hiện nay đã trở thành chuẩn mực công nghiệp thống nhất cho cả 3 trụ cột (Logs, Metrics, Traces), định nghĩa chuẩn truyền tải OTLP.

Để bật tracing và xuất dữ liệu ra OTel Collector, chúng ta chỉ cần khai báo các thư viện phụ thuộc (dependencies) và cấu hình trong `application.yml` mà không phải sửa code logic:

Listing 11.4: Khai báo thư viện phụ thuộc cho tracing (`pom.xml`)

```

<!-- Spring Boot 3 Actuator -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<!-- Micrometer OTel Bridge -->
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
<!-- OTLP Exporter -->
<dependency>
  <groupId>io.opentelemetry</groupId>
  <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>

```

Tuy nhiên, Distributed Tracing chỉ phát huy tối đa hiệu quả khi được kết hợp với Logs. Tracing giúp định vị ‘nút thắt’ (bottleneck), còn Logs giải thích nguyên nhân gốc rễ (root cause).

Tracing Complements Logging

“Tracing xác định đường đi và thời gian xử lý của request, trong khi Logging giải thích nguyên nhân gây ra vấn đề. Sử dụng trace để nhận diện điểm nghẽn (bottleneck) và dùng logs để hiểu nguyên nhân gốc rễ (root cause). Hai công cụ này bổ trợ cho nhau và không thể thay thế lẫn nhau.”

— Tổng hợp từ Richardson [2a, Ch.11] và Newman [4a, Ch.8]

Sau phần thư viện phụ thuộc, việc còn lại là khai báo nơi xuất dữ liệu (OTLP exporter) và tỉ lệ lấy mẫu trong `application.yml` (Listing 11.5):

Listing 11.5: Cấu hình OTLP Exporter và Sampling (`application.yml`)

```

management:
  endpoints:
    web:
      exposure:
        include: health,prometheus # Expose metrics
  tracing:
    sampling:
      probability: 1.0 # Sample 100% (cho môi trường dev/staging)
  otlp:
    tracing:
      endpoint: http://otel-collector:4318/v1/traces

```

Với cấu hình này, Spring Boot **tự động** truyền trace context qua HTTP headers theo chuẩn **W3C Trace Context** (header `traceparent`, thay cho chuẩn B3 cũ). Để truyền qua Kafka, cần bật observation ở *cả hai phía*: `spring.kafka.template.observation-enabled=true` cho producer (`KafkaTemplate`) và `spring.kafka.listener.observation-enabled=true` cho consumer — thiếu phía producer thì trace sẽ đứt đoạn tại broker. Không cần viết code tracing thủ công cho phần lớn bài toán.

11.3.6. Kiến trúc triển khai Observability Pipeline

Khi hệ thống lớn lên, ứng dụng không nên gửi trực tiếp dữ liệu đến Jaeger/Prometheus. Có 3 mô hình triển khai:

1. **Direct SDK**: Ứng dụng gửi trực tiếp dữ liệu đến hệ thống lưu trữ (tight coupling, dễ gây nghẽn mạng).
2. **Local Agent**: Ứng dụng gửi dữ liệu nội bộ tới agent chạy dưới dạng DaemonSet; agent tự quản lý bộ đệm và gom lô (batching).
3. **Centralized Collector**: Tất cả Agent gửi về một cụm Gateway Collector trung gian để che dữ liệu cá nhân (PII), lọc và định tuyến trước khi lưu trữ.

⚠ Nút thắt OTel Collector

Trong mô hình Centralized Collector, cụm Collector là “Single Point of Processing”.

- **OOM (Out of Memory)**: Rất dễ xảy ra nếu dùng **Tail-based sampling**, vì Collector phải lưu trữ tạm toàn bộ Span của mọi request trên RAM để đợi quyết định lưu trữ hay loại bỏ ở bước cuối cùng.
- **CPU Bound & Network Saturation**: Do phải phân tích (parse) log/traces và lượng dữ liệu khổng lồ.

Biện pháp giảm thiểu: Triển khai kiến trúc phân tầng (tiered: Agent ở biên, Gateway ở trung tâm), dùng Apache Kafka ở giữa làm bộ đệm hấp thụ các đợt lưu lượng tăng đột biến, và đặt **chính sách lưu giữ log** (retention: chỉ lưu log nóng trong 7-14 ngày trên Loki, rồi lưu trữ sang S3 để tối ưu chi phí lưu trữ).

11.4. Metrics, SLI/SLO và chiến lược cảnh báo

Hệ thống cần các chỉ tiêu chất lượng rõ ràng để cảnh báo đội ngũ kỹ thuật kịp thời — trước khi sự cố kịp làm gián đoạn việc học của sinh viên.

11.4.1. Metrics — đo lường hành vi theo thời gian

Trước khi thiết lập cảnh báo dựa trên metrics, cần tránh một lỗi thiết kế phổ biến trong thu thập dữ liệu chuỗi thời gian:

Một sai lầm kinh điển của các nhóm mới làm quen với microservices là đưa `userId` hoặc `submissionId` vào làm “label” (tag) cho Prometheus metrics. Metrics được thiết kế cho **Low Cardinality** (số lượng tag hữu hạn, vd: `status=200, method=GET`). Nếu nhóm phát triển gắn `userId` làm nhãn, cơ sở dữ liệu chuỗi thời gian (TSDB) sẽ sinh ra hàng triệu time-series khác nhau, ngốn hết bộ nhớ và làm sập hệ thống giám sát. Để tra cứu theo `userId`, hãy dùng Logs hoặc Traces.

⚠ Cardinality Explosion

Logs ghi lại *sự kiện đã xảy ra*. Metrics đo lường *xu hướng theo thời gian* — quan trọng cho việc **phát hiện sớm** (early detection) vấn đề trước khi người dùng báo lỗi.

Richardson trong [2a, Ch.11] mô tả bốn observability pattern cần triển khai:

Bảng 11.7: Bốn observability pattern cần triển khai

Pattern	Mô tả	Ví dụ KBM
Health Check API	Endpoint <code>/health</code> cho hệ thống điều phối (orchestrator) biết dịch vụ đang hoạt động hay đã ngừng (up/down)	Eureka lưu và công bố trạng thái từ health check; LoadBalancer dựa vào đó để chọn instance
Application metrics	Metrics ở mức nghiệp vụ	Tần suất nộp bài, thời gian chấm, tỷ lệ lỗi
Infrastructure metrics	Metrics ở mức hệ thống	CPU, memory, DB connections, Kafka consumer lag
Audit logging	Ghi lại hành vi người dùng để đáp ứng yêu cầu tuân thủ (compliance)	Đăng nhập, thay đổi vai trò, truy cập dữ liệu

11.4.2. Bốn loại Metrics cơ bản của Prometheus

Trước khi áp dụng RED/USE, lập trình viên cần hiểu 4 kiểu dữ liệu đo lường:

1. **Counter:** Bộ đếm chỉ tăng lên (vd: Tổng số requests). Tốt để tính Rate.
2. **Gauge:** Giá trị có thể tăng/giảm (vd: mức dùng CPU, số kết nối đang mở). Tốt để tính Saturation.
3. **Histogram:** Nhóm dữ liệu vào các “bucket” (vd: Số request xong trong <100ms, <500ms). Nên dùng cho phân vị độ trễ (P99) vì phía server tính lại và *gộp* được số liệu qua nhiều instance.
4. **Summary:** Tính sẵn quantile ở phía client — chính xác cho từng instance nhưng không gộp được giữa các replica, nên ít phù hợp với hệ phân tán nhiều bản sao.

💡 Exemplars: cầu nối Metrics & Traces

Công nghệ hiện đại cho phép nhúng trực tiếp một **traceId** đại diện vào trong các bucket của Prometheus Histogram (gọi là **Exemplars**). Khi quan sát thấy một đỉnh trễ bất thường trên Grafana, kỹ sư có thể nhấp trực tiếp vào điểm dữ liệu đó để chuyển hướng đến Jaeger và phân tích chính xác trace gây ra sự chậm trễ.

Để quy chuẩn hóa việc đo lường trạng thái hệ thống, ngành công nghiệp thường sử dụng hai khuôn mẫu tiêu chuẩn: RED (cho dịch vụ) và USE (cho tài nguyên).

11.4.3. Hai khung đo lường: RED và USE

- **RED (dùng cho service phục vụ request):** Bao gồm **Rate**, **Errors**, và **Duration**. Ví dụ: tần suất nộp bài là 50 req/min, tỷ lệ lỗi là 2%, độ trễ P99 là 3.2s.
- **USE (dùng cho tài nguyên hạ tầng):** Bao gồm **Utilization**, **Saturation**, và **Errors**. Ví dụ: CPU là 45%, DB connection pool là 8/10 active, Disk I/O errors là 0.

Kết hợp RED (cho services) + USE (cho resources) để giám sát cả hai khía cạnh của hệ thống. Metrics phát hiện “độ trễ nộp bài tăng” (RED), điều tra cho thấy “DB connection pool bị bão hòa” (USE) → nguyên nhân gốc rễ: rò rỉ kết nối (connection leak).

11.4.4. SLI/SLO/SLA — từ metrics đến cam kết chất lượng

Metrics chỉ hữu ích khi biết **ngưỡng nào là chấp nhận được**. Khung SLI/SLO/SLA cung cấp ngôn ngữ chung:

💡 Thư viện trường học

- **SLI (Chỉ số thực tế)**: Thời gian thực tế thủ thư dùng để xếp một cuốn sách trả lại lên kệ (vd: 1.5 giờ).
- **SLO (Mục tiêu nội bộ)**: KPI mà giám đốc Thư viện đặt ra cho nhân viên: “99% sách trả lại phải được xếp lên kệ trong vòng 2 giờ”.
- **SLA (Cam kết đền bù)**: Cam kết trong Sổ tay sinh viên: “Nếu sách người mượn trả chưa được hệ thống cập nhật trong ngày, nhà trường cam kết sẽ miễn trừ mọi khoản phạt trễ hạn tự động phát sinh”.

Để đảm bảo chất lượng vận hành, hệ sinh thái đo lường được chia thành ba khái niệm chính:

- **SLI (Service Level Indicator)**: Là metric cụ thể được hệ thống đo lường trực tiếp để đánh giá chất lượng dịch vụ trong thực tế. Trong KBM, ví dụ tiêu biểu là đo “Độ trễ P99 của luồng nộp bài”.
- **SLO (Service Level Objective)**: Là mục tiêu nội bộ mà đội ngũ kỹ thuật tự đặt ra đối với một SLI nhất định. Ví dụ: “99% bài nộp phải trả kết quả trong vòng chưa tới 5 giây”.
- **SLA (Service Level Agreement)**: Là những cam kết chính thức với khách hàng, có đi kèm với các hình phạt tài chính hoặc hậu quả pháp lý rõ ràng nếu hệ thống bị vi phạm. Ví dụ: “Hệ thống cam kết tính sẵn sàng (availability) đạt mức 99.5% tổng thời gian trong một học kỳ”.

Các khái niệm trên phân biệt rõ ba mức độ hay bị nhầm lẫn: SLI là con số đo được, SLO là mục tiêu nội bộ mà nhóm phát triển tự đặt, SLA là cam kết có ràng buộc pháp lý với bên ngoài. Hình dưới minh họa mối quan hệ lồng nhau giữa chúng — SLI nằm bên trong SLO, SLO nằm bên trong SLA.



Hình 11.7: Mối quan hệ giữa SLI, SLO và SLA

Với KBM — hệ thống giáo dục nội bộ phục vụ sinh viên của trường — nhóm phát triển không cần SLA chính thức vì không có hợp đồng thương mại. Tuy nhiên, SLO nội bộ vẫn rất quan trọng: chúng đặt ngưỡng để nhóm phát triển biết khi nào cần ưu tiên khắc phục lỗi, khi nào chấp nhận được. Bảng dưới đề xuất bốn SLO phù hợp với bối cảnh KBM:

Bảng 11.8: SLO đề xuất cho KBM

SLI	SLO	Lý giải
Độ trễ nộp bài P99	< 5 giây	Sinh viên phải chờ đợi lâu dẫn đến tâm lý thất vọng và ngừng sử dụng hệ thống
Tính sẵn sàng trong kỳ thi	99.9% (~8.7h downtime/năm)	Kỳ thi có thời gian giới hạn — downtime ảnh hưởng trực tiếp điểm
Độ chính xác chấm điểm	100%	Sai sót ảnh hưởng trực tiếp đến kết quả học tập của sinh viên — đây là chỉ số không chấp nhận sai lệch
Tỷ lệ lỗi API	< 1%	Tần suất lỗi cao sẽ làm giảm mức độ tin cậy của người dùng đối với hệ thống

11.4.5. Chiến lược cảnh báo

Tuy nhiên, nếu đội ngũ kỹ sư cấu hình cảnh báo quá nhạy cảm hoặc sai cách, họ sẽ nhanh chóng rơi vào trạng thái “mệt mỏi vì cảnh báo” (**alert fatigue**). Hiện tượng này xảy ra khi có quá nhiều cảnh báo giả (false positives) liên tục phát sinh, khiến các kỹ sư on-call dần sinh ra tâm lý chủ quan và bỏ qua ngay cả những sự cố thực sự nghiêm trọng. Để chống lại alert fatigue, một cơ chế quan trọng cần áp dụng là **gom nhóm cảnh báo** (alert grouping) — thay vì gửi 100 thông báo riêng lẻ khi một dịch vụ ngừng hoạt động (down) kéo theo hàng loạt lỗi liên quan, hệ thống sẽ gom chúng lại thành một thông báo duy nhất theo nguyên nhân gốc rễ.

Bảng 11.9: Chiến lược cảnh báo theo mức độ nghiêm trọng

Mức độ	Khi nào	Kênh	Hành động	Ví dụ
Critical	SLO bị vi phạm, ảnh hưởng đến người dùng ngay lập tức	PagerDuty, gọi điện	Kỹ sư trực hệ thống (on-call) phải xử lý ngay	Dịch vụ Judge ngừng hoạt động, nộp bài thất bại
Warning	Gần vi phạm SLO, cần chú ý	Slack, email	Xem xét trong giờ làm việc	Độ trễ P99 đạt 80% ngưỡng
Info	Bất thường nhưng không ảnh hưởng người dùng	Chỉ hiển thị trên dashboard	Xem xét sau hoặc phân tích định kỳ	Kafka consumer lag tăng nhẹ

Ba tầng cảnh báo ở trên giúp tránh tình trạng “mọi sự cố đều được coi là critical”. Khi áp dụng thực tiễn, việc xây dựng tiêu chí cảnh báo cần gắn kết trực tiếp với trải nghiệm người dùng (thông qua SLO), thay vì dựa trên chỉ số kỹ thuật thuần túy. Chẳng hạn, nếu CPU tăng cao nhưng độ trễ vẫn trong giới hạn và người dùng không bị ảnh hưởng, thì không cần đánh thức kỹ sư on-call vào lúc 3 giờ sáng.

Cảnh báo theo triệu chứng, không theo nguyên nhân

“Cảnh báo khi SLO có nguy cơ bị vi phạm, không phải khi một metric thay đổi. CPU 90% nhưng độ trễ bình thường? Không cần cảnh báo. CPU 50% nhưng độ trễ vượt SLO? Cảnh báo ngay.”

— Nguyên tắc SRE, dựa trên Google SRE book

11.5. Chiến lược xử lý lỗi — nhất quán giữa các service

Một hệ thống nhiều thành phần thường gặp tình trạng mỗi dịch vụ trả về một định dạng báo lỗi khác nhau. Để tránh gây nhầm lẫn cho ứng dụng frontend và sinh viên, hệ thống cần chuẩn hóa cách xử lý lỗi.

Vấn đề: mỗi service trả về định dạng lỗi khác nhau. Khi hệ thống bao gồm nhiều dịch vụ do các nhóm khác nhau phát triển, định dạng phản hồi lỗi (error response) rất dễ mất tính nhất quán. Dịch vụ A trả về lỗi trong trường `description`, Dịch vụ B lại sử dụng trường `message`, Gateway trả định dạng hoàn toàn khác. Ứng dụng khách (client/frontend) phải xử lý nhiều định dạng — mã nguồn phình phức tạp và trải nghiệm người dùng (UX) kém đi. Do đó cần thiết lập nguyên tắc chuẩn hóa.

11.5.1. Nguyên tắc thiết kế xử lý lỗi

Richardson trong [2a, Ch.11] và Newman trong [4a, Ch.8] đề xuất các nguyên tắc xử lý lỗi trong microservices:

1. Bộ xử lý ngoại lệ tập trung (centralized exception handler) — Mỗi service có một handler tập trung (trong Spring Boot: `@ControllerAdvice`) xử lý mọi ngoại lệ → chuyển thành định dạng chuẩn. Đặt trong thư viện dùng chung để mọi service cùng sử dụng.

2. Phân loại mã lỗi (error code taxonomy) — Phân loại lỗi theo namespace để client và hệ thống giám sát hiểu nhanh:

Bảng 11.10: Phân loại hệ thống mã lỗi (`errorCode` dạng chuỗi có namespace)

Namespace	Nhóm	Ví dụ <code>errorCode</code>
AUTH_*	Xác thực & phân quyền	AUTH_UNAUTHORIZED, AUTH_TOKEN_EXPIRED
NOT_FOUND_*	Không tìm thấy tài nguyên	NOT_FOUND_QUESTION, NOT_FOUND_CONTEST
BIZ_*	Vi phạm quy tắc nghiệp vụ	BIZ_CONTEST_NOT_STARTED, BIZ_SUBMIT_LIMIT
INFRA_*	Hạ tầng / phụ thuộc	JUDGE_UNAVAILABLE, KAFKA_SEND_FAILED (giữ tên phụ thuộc cho dễ đọc)
INTERNAL_*	Lỗi nội bộ	INTERNAL_UNEXPECTED

3. Problem Details (RFC 9457) — Mọi phản hồi lỗi dùng cùng cấu trúc Problem Details for HTTP APIs (RFC 9457), với các extension nội bộ KBM (`errorCode`, `traceId`):

Listing 11.6: Error response theo RFC 9457 Problem Details

```
HTTP/1.1 503 Service Unavailable
Content-Type: application/problem+json

{
  "type": "urn:kbm:problem:judge-unavailable",
  "title": "Judge service unavailable",
  "status": 503,
  "detail": "Judge service did not respond within timeout.",
  "instance": "urn:kbm:problem-instance:req-a1b2c3",
  "errorCode": "JUDGE_UNAVAILABLE",
  "traceId": "abc-123-def-456"
}
```

4. Không để lộ chi tiết nội bộ — Stack trace, câu truy vấn SQL, đường dẫn nội bộ không bao giờ xuất hiện trong phản hồi lỗi. Ghi log chi tiết ở phía server (kèm traceId để đối chiếu), chỉ trả thông báo chung cho client.

Mối liên hệ giữa xử lý lỗi và observability. Trong thực tế, các mã lỗi tập trung không chỉ phục vụ client — chúng là **nguồn dữ liệu quan trọng cho metrics và cảnh báo**:

- Đếm lỗi theo mã → phát hiện xu hướng: **JUDGE_UNAVAILABLE** tăng đột biến → cảnh báo
- Gom lỗi theo nhóm → dashboard: lỗi xác thực, lỗi nghiệp vụ, lỗi hạ tầng
- Tỷ lệ lỗi theo service → SLI cho độ tin cậy

① Định dạng lỗi không nhất quán trong KBM

KBM sử dụng `@ControllerAdvice` với `ErrorCode` enum trong shared library — đúng pattern. Tuy nhiên, **Gateway** (Spring Cloud Gateway dùng WebFlux) xử lý lỗi JWT bên ngoài `@ControllerAdvice` (WebMVC pattern). Kết quả: Gateway trả error dạng `{ "message": "..."}` trong khi các services trả `{ "description": "..."}` — không theo RFC 9457 và tên trường khác nhau. Ứng dụng giao diện (frontend) phải xử lý đồng thời cả hai định dạng này.

Lộ trình chuyển đổi (Migration path): (1) Tạo `GatewayExceptionHandler` riêng cho WebFlux, chuẩn hóa response theo `application/problem+json` (RFC 9457), (2) Đảm bảo tất cả error responses — kể cả từ Gateway — dùng cùng Problem Details schema với `errorCode` (chuỗi) và `traceId`, (3) Loại bỏ các trường không thuộc hợp đồng truyền dữ liệu chuẩn (`timestamp`, `description`, `message`).

11.6. Health Checks và Service Discovery

Service Discovery và API Gateway chỉ có thể tự động ngắt routing khỏi dịch vụ gặp sự cố khi có căn cứ: đó là vai trò của Health Checks.

11.6.1. Health Checks — nền tảng cho tự động hóa

Richardson trong [2a, Ch.11] mô tả **Health Check API** là observability pattern cơ bản nhất: mỗi dịch vụ cần cung cấp (expose) một endpoint `/health` để các công cụ điều phối nhận biết trạng thái hoạt động của nó.

Health checks phân loại theo mục đích:

Bảng 11.11: Ba loại Health Checks

Loại	Câu hỏi trả lời	Ý nghĩa cho orchestrator	Ví dụ KBM
Liveness	“Tiến trình còn chạy?”	Khởi động lại container nếu tiến trình không phản hồi	JVM còn hoạt động, không bế tắc (deadlock)
Readiness	“Sẵn sàng nhận traffic?”	Ngừng chuyển traffic nếu dịch vụ chưa sẵn sàng	Database connected, Kafka connected
Startup	“Đã khởi tạo xong?”	Chờ quá trình khởi động hoàn tất mới kiểm tra liveness	Đã chạy migration lược đồ, đã làm nóng cache

Trong KBM, health checks tham gia vào định tuyến theo một chuỗi trách nhiệm rõ ràng: instance Core Service báo **DOWN** qua endpoint health → **Eureka** (Service Registry) cập nhật trạng thái trong danh bạ → Spring Cloud LoadBalancer ở Gateway loại instance đó khỏi danh sách lựa chọn và

chuyển traffic sang instance khỏe mạnh. DevOps Lab bổ sung health/readiness ở runtime layer để router chỉ chuyển traffic đến workspace sẵn sàng. Còn việc *khởi động lại* instance hòng thuộc về tầng orchestrator/supervisor (chính sách restart của Docker, Kubernetes) — kết hợp cả hai tầng mới tạo thành bức tranh **self-healing** hoàn chỉnh không cần can thiệp thủ công.

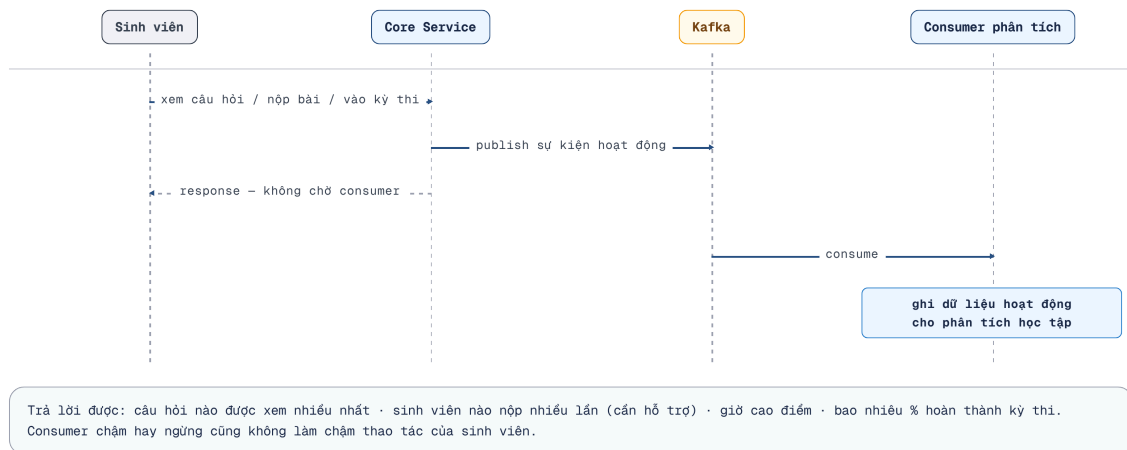
⚠ Eureka không tự suy ra trạng thái health của service

Một cấu hình dễ bỏ sót: mặc định Eureka client chỉ gửi *heartbeat* để báo tiến trình còn sống, nên một service vẫn được công bố **UP** dù endpoint health đã **DOWN** (mất kết nối database chẳng hạn). Traffic khi đó tiếp tục được chuyển đến một instance không phục vụ được. KBM bật `eureka.client.healthcheck.enabled=true` để trạng thái từ Spring Boot Actuator được truyền lên registry. Khi bật, cần cân nhắc chính health model của hệ thống: nếu endpoint health kiểm tra quá sâu vào các phụ thuộc bên ngoài, một sự cố nhất thời có thể khiến hàng loạt instance bị rút khỏi danh bạ cùng lúc (xem thêm phần phân biệt liveness và readiness ở cuối chương).

11.7. User Activity Tracking — Observability ở mức nghiệp vụ

Ba trụ cột (logs, metrics, traces) giúp quan sát **hệ thống kỹ thuật**. Nhưng với KBM — một LMS — cần thêm: quan sát **hành vi học tập**. Sinh viên đang học gì? Bài nào khó nhất? Khi nào sinh viên hoạt động nhiều nhất?

KBM đã triển khai **User Activity Tracking** qua Kafka pipeline: khi sinh viên xem câu hỏi, nộp bài, hoặc tham gia kỳ thi, Core Service sẽ phát các sự kiện theo dõi lên Kafka topic. Consumer sẽ tiếp nhận và ghi dữ liệu — quá trình này diễn ra hoàn toàn bất đồng bộ và không chặn (non-blocking), do đó không làm ảnh hưởng đến thời gian phản hồi của ứng dụng.



Hình 11.8: User Activity Tracking pipeline qua Kafka

Dữ liệu thu được từ pipeline này phục vụ nhiều mục đích, tóm tắt ở Bảng 11.12.

Bảng 11.12: Ứng dụng của User Activity Tracking

Dữ liệu	Câu hỏi trả lời được	Ứng dụng
Mẫu xem bài	“Câu hỏi nào được xem nhiều nhất?”	Đề xuất bài tập phù hợp
Mẫu nộp bài	“Sinh viên nào nộp bài nhiều lần?”	Phát hiện sinh viên cần hỗ trợ

Dữ liệu	Câu hỏi trả lời được	Ứng dụng
Mẫu thời gian	“Giờ cao điểm là khi nào?”	Mở rộng hạ tầng phù hợp
Hành vi trong kỳ thi	“Bao nhiêu % sinh viên hoàn thành kỳ thi?”	Đánh giá hiệu quả giảng dạy

Đây là điểm giao giữa observability kỹ thuật và **phân tích đặc thù miền** (domain-specific analytics) — một giá trị riêng mà observability mang lại cho LMS, vượt xa việc chỉ phục vụ đội ngũ vận hành.

11.8. Chaos Engineering — kiểm tra khả năng phục hồi (resilience) chủ động

Observability cho phép **phát hiện** sự cố. Nhưng làm sao biết hệ thống sẽ phản ứng thế nào *trước* khi sự cố thật xảy ra? **Chaos Engineering** — phương pháp được Netflix phổ biến qua Chaos Monkey (2010) — trả lời câu hỏi này bằng cách **chủ động tiêm lỗi** (inject failures) vào production/staging và quan sát phản ứng. Cần lưu ý rằng Chaos Engineering là định hướng ở giai đoạn trưởng thành cao (từ Cấp 4 trở lên), với KBM, trước mắt nên bắt đầu bằng giả lập lỗi thủ công để kiểm chứng tính bền bỉ.

Nguyên tắc (theo *Principles of Chaos Engineering* — principlesofchaos.org):

1. **Xác định trạng thái ổn định (steady state)** — hệ thống hoạt động bình thường trông như thế nào? (độ trễ < 200ms, tỷ lệ lỗi < 0.1%, throughput > 100 req/s)
2. **Đặt giả thuyết** — “Nếu Judge Service ngừng hoạt động (down), bài nộp vẫn được xếp hàng và xử lý khi Judge khởi động lại”
3. **Tiêm lỗi** — Dừng đột ngột (kill) container Judge Service
4. **Quan sát** — Trạng thái ổn định có bị phá vỡ không? Hệ thống phục hồi trong bao lâu?
5. **Khắc phục điểm yếu** — Nếu trạng thái ổn định bị phá vỡ, cần sửa lỗi trên hệ thống (fix system) thay vì điều chỉnh lại kịch bản kiểm thử (fix test)

💡 Diễn tập phòng cháy chữa cháy

Chaos Engineering giống như nhà trường cố tình bấm còi báo “Diễn tập phòng cháy chữa cháy” đột xuất ở Ký túc xá lúc 9 giờ tối. Mục đích là chủ động xem sinh viên có thoát hiểm kịp trong 5 phút không, cửa thoát hiểm có mở không — thay vì đợi đến khi có hỏa hoạn thật mới phát hiện ra cửa đã bị rỉ sét không mở được.

Tương tự như việc ban quản lý ký túc xá chuẩn bị nhiều kịch bản diễn tập khác nhau (chập điện, rò rỉ nước, hỏa hoạn), kỹ sư hệ thống cũng có các kịch bản tiêm lỗi đa dạng để kiểm thử độ bền bỉ của nền tảng:

Bảng 11.13: Phân loại kỹ thuật tiêm lỗi trong Chaos Engineering

Loại lỗi được tiêm	Ví dụ	Công cụ
Lỗi tiến trình	Kill container/service	Chaos Monkey, docker stop
Lỗi mạng	Delay, packet loss, partition	Toxiproxy, Gremlin
Cạn tài nguyên	CPU 100%, disk full, memory leak	stress-ng, Gremlin
Lỗi phụ thuộc	Database down, Kafka unavailable	Litmus, thủ công

Bốn loại lỗi được tiêm trên phù gần hết các miền lỗi (failure domain) trong hệ thống phân tán. Tuy nhiên, nhóm phát triển không cần chạy tất cả cùng lúc — nên bắt đầu từ ba thí nghiệm đơn

giản nhất, vừa tầm với hạ tầng Docker Compose hiện tại của KBM, và đã đủ kiểm chứng resilience patterns đã thiết kế ở các chương trước:

- **Kill Judge Service:** Giả thuyết đặt ra là các bài nộp của sinh viên sẽ tự động được xếp hàng an toàn trong Kafka và xử lý tiếp khi service khởi động lại. Kết quả mong đợi là Kafka retention sẽ giữ toàn bộ thông điệp và Judge bắt kịp tiến độ ngay khi dịch vụ được phục hồi.
- **Network delay 500ms giữa Gateway↔Core:** Giả thuyết cho rằng sinh viên sẽ trải nghiệm tốc độ phản hồi chậm hơn đôi chút nhưng hệ thống hoàn toàn không trả về lỗi. Kết quả mong đợi (khi đã có circuit breaker ở Giai đoạn 3, Ch.10) là timeout và circuit breaker ngăn lỗi dây chuyền (cascading failure); chạy thí nghiệm này sớm hơn sẽ cho thấy đúng khoảng trống đó.
- **PostgreSQL restart:** Giả thuyết là Core Service có khả năng tự động kết nối lại cơ sở dữ liệu. Kết quả mong đợi là Connection pool (HikariCP) tự động xử lý toàn trình việc kết nối lại mà không yêu cầu khởi động lại ứng dụng.

Điều quan trọng là bắt đầu — không cần công cụ phức tạp. Docker CLI đã là đủ cho các thí nghiệm chaos đầu tiên, và giá trị mang lại là sự tự tin rằng hệ thống thực sự phục hồi được.

💡 Chaos Engineering từ staging

Không cần Chaos Monkey hoặc công cụ phức tạp. Bắt đầu đơn giản: `docker stop kbm-judge-sql` trên staging → quan sát logs → xem hệ thống có phục hồi không. Đây đã là chaos engineering. Khi nhóm đã quen, nâng lên: các thí nghiệm chaos tự động chạy trong CI/CD pipeline.

11.8.1. Case study thực tế: Netflix Simian Army

Netflix là người tiên phong của Chaos Engineering. Năm 2010, sau khi chuyển từ trung tâm dữ liệu sang AWS, Netflix tạo **Chaos Monkey** — công cụ tự động tắt ngẫu nhiên các EC2 instance trong production. Mục đích: nếu hệ thống chịu được các sự cố ngẫu nhiên hằng ngày, nó sẽ chịu được sự cố thật.

Chaos Monkey phát triển thành **Simian Army** — tập hợp các công cụ chuyên biệt:

Bảng 11.14: Netflix Simian Army

Công cụ	Chức năng	Bài học
Chaos Monkey	Kill ngẫu nhiên các instance	Services phải phi trạng thái (stateless), tự khởi động lại được
Chaos Kong	Giả lập sự cố toàn bộ region	Cơ chế chuyển đổi dự phòng đa vùng (multi-region failover) phải hoạt động
Latency Monkey	Tiêm độ trễ mạng (network delay)	Timeout + circuit breaker phải được cấu hình đúng
Conformity Monkey	Kiểm tra instance tuân thủ các giải pháp khuyến nghị	Tự động hóa thay cho kiểm tra thủ công

Kết quả: Netflix duy trì độ sẵn sàng cao (mục tiêu 99.99%) khi phục vụ hơn 200 triệu người đăng ký (2021). Casey Rosenthal — người dẫn dắt nhóm Chaos Engineering của Netflix, sau này đồng sáng lập Verica — cùng Nora Jones là đồng tác giả *Chaos Engineering: System Resiliency in Practice* (O'Reilly, 2020).

Bài học cho hệ thống nhỏ: LMS không cần Simian Army. Nhưng nguyên tắc cốt lõi áp dụng: nếu đội ngũ phát triển ngại việc vô hiệu hóa đột ngột (kill) một dịch vụ, đó là dấu hiệu hệ thống thiếu khả năng phục hồi. Bắt đầu thủ công (**docker stop**), tiến tới tự động (CI/CD chaos stage).

Nguồn: Netflix Technology Blog, “The Netflix Simian Army,” 2011 (netflixtechblog.com). Casey Rosenthal et al., *Chaos Engineering: System Resiliency in Practice*, O’Reilly, 2020.

11.9. Case study: KBM

Phần này phân tích hiện trạng giám sát của KBM và vạch lộ trình nâng cấp tương ứng.

11.9.1. Hiện trạng — đánh giá theo mô hình trưởng thành

Hình 11.9 đánh giá hiện trạng observability của KBM theo từng thành phần.



Hình 11.9: Đánh giá hiện trạng Observability của KBM

Đánh giá tổng thể: Cấp 2 đang tiến lên Cấp 3 — KBM đã có Actuator/health ở một số services và nền tảng Prometheus/Grafana cho một phần hệ thống, nhưng vẫn cần chuẩn hóa correlation ID, quy trình thu thập log và truy vết (tracing) xuyên suốt ranh giới các dịch vụ. Khi yêu cầu nộp bài phản hồi chậm hoặc thất bại, lập trình viên không nên phải mở nhật ký của từng container và ghép timestamp thủ công.

Bảng 11.15: Mức độ trưởng thành observability phân theo thành phần

Thành phần	Hiện trạng	Cấp độ trưởng thành
Logging	Java services có log qua MDC ở một số nơi; DevOps Lab dùng zerolog; cần gom tập trung	● Cấp 2
Metrics	Actuator/health + Prometheus/Grafana ở một phần hệ thống	● Cấp 2
Tracing	Distributed tracing (W3C Trace Context) chưa triển khai; Correlation ID có nền tảng nhưng chưa chuẩn hóa đầu cuối	● Cấp 2
Xử lý lỗi	@ControllerAdvice + ErrorCode tốt, Gateway chưa nhất quán	● Cấp 3
Theo dõi hành vi người dùng	Bắt đồng bộ qua Kafka — tốt	● Cấp 3

11.9.2. Phân tích theo business context

KBM phục vụ sinh viên học SQL, mạng, DevOps và các hoạt động LMS. Điểm đặc biệt là hệ thống có nhiều chế độ hoạt động khác nhau — mỗi chế độ có mẫu lưu lượng, yêu cầu độ trễ và mức chịu

lỗi riêng. Chiến lược observability không thể áp dụng một khuôn cho tất cả: chế độ thi cần cảnh báo thời gian thực, chế độ luyện tập chỉ cần giám sát cơ bản. Bảng dưới phân tích chi tiết từng chế độ:

Bảng 11.16: Các chế độ hoạt động chính của LMS

Chế độ	Đặc điểm	SLO cần thiết	Observability cần
Chế độ luyện tập	Sinh viên tự luyện tập, traffic thấp, không có hạn chót	Độ trễ < 10s là được, downtime chấp nhận được	Giám sát cơ bản là đủ
Chế độ thi	100+ sinh viên nộp bài đồng thời, có hạn chót, bảng xếp hạng thời gian thực	Độ trễ < 5s, availability 99.9%	Metrics + cảnh báo bắt buộc
DevOps Lab runtime	Runtime của workspace/container/mạng biến động	Readiness chính xác, cô lập lỗi	Promtail/Loki/Grafana + runtime metrics

Trong chế độ thi, **không có observability = thiếu tầm nhìn hệ thống (flying blind)**:

- Judge Service quá tải → bài nộp dồn hàng đợi → bảng xếp hạng lỗi thời → sinh viên phàn nàn
- Không có metrics tùy biến → không biết thời gian chấm tăng cho đến khi sinh viên báo
- Không có tracing → không biết nút thắt ở Judge hay ở Kafka consumer

Observability chưa chuẩn hóa đầu cuối

KBM đã có một số mảnh observability đúng hướng: Actuator/Prometheus cho Java services, **X-Request-ID**/MDC ở shared layer, zerolog ở Go services, và PLG stack (Promtail/Loki/Grafana) cho DevOps Lab. Tuy nhiên, hệ thống vẫn còn thiếu sự đồng nhất trên toàn chuỗi giao dịch.

Khoảng trống lớn nhất hiện tại là **chuẩn hóa đầu cuối**: cùng một request hoặc batch cần đi qua Gateway → service → Kafka/DB queue → Judge/runtime mà vẫn giữ được các identifier (correlation ID cho workflow, trace ID cho distributed trace, request ID cho ingress) nguyên vẹn. Việc thiếu hụt này khiến quá trình gỡ rối trở nên gián đoạn và khó theo dõi toàn bộ luồng xử lý thực tế.

Để khắc phục, chúng ta cần một **lộ trình chuyển đổi** (tăng dần — mỗi giai đoạn đều mang lại giá trị ngay, mục tiêu: Cấp 3) như sau:

- **Giai đoạn 1 — Log có cấu trúc + Request ID** (chi phí triển khai thấp, tác động cao): Chuẩn hóa JSON logging cho các dịch vụ Java và zerolog field names cho Go, đồng thời chuẩn hóa **X-Request-ID** (Gateway đã sinh) lan truyền qua HTTP headers (Kafka dùng **correlationId** trong message envelope). Giá trị nhận được ngay lập tức là khả năng gỡ rối chéo dịch vụ chỉ qua việc tìm kiếm một request ID duy nhất.
- **Giai đoạn 2 — Gom log tập trung** (chi phí triển khai trung bình): Mở rộng stack log đang chạy ở DevOps Lab (Loki + Grafana, thu gom bằng Grafana Alloy hoặc Promtail) sang LMS core. Việc cấu hình pipeline thu thập log tập trung sẽ chấm dứt ngay thói quen SSH vào từng container để đọc log thủ công.
- **Giai đoạn 3 — Distributed Tracing** (chi phí triển khai trung bình): Tích hợp Micrometer Tracing và OpenTelemetry vào mỗi dịch vụ, cùng với Jaeger làm backend, cho phép chúng ta nhìn rõ chính xác điểm nghẽn mạng — liệu sự chậm trễ đến từ Judge Service, Kafka hay Database.
- **Giai đoạn 4 — Metrics nghiệp vụ + Cảnh báo** (chi phí triển khai trung bình): Mở rộng Prometheus/Grafana hiện có ra mọi service, thêm metrics nghiệp vụ (tần suất nộp bài, thời

gian chấm), từ đó thiết lập các cảnh báo dựa trên SLO thực tế như độ trễ nộp bài. Các cảnh báo sẽ được phân cấp hợp lý để gửi thông báo tức thời nếu Judge Service ngừng hoạt động, nhưng chỉ hiển thị độ trễ bất thường dưới dạng cảnh báo nhẹ.

Những cạm bẫy về Observability. Hệ thống giám sát thiết lập sai cách sẽ tự sinh nhiễu (noise), lãng phí tài nguyên và làm giảm độ nhạy bén của đội ngũ vận hành — độ chi tiết của dữ liệu phải được cân với chi phí. Những sai lầm phổ biến nhất:

1. **Log không cấu trúc, thiếu request/correlation ID** — Mỗi service ghi log dạng văn bản tự do riêng, không có ID xuyên suốt một request; khớp log bằng timestamp sai lệch ngay khi có xử lý đồng thời → debug như “mò kim đáy bể”. *Phòng tránh:* log có cấu trúc (JSON) + request ID (HTTP) và correlation ID (business workflow) khởi tạo tại Gateway và lan truyền qua mọi service.
2. **Logging quá nhiều hoặc quá ít** — Ghi toàn bộ request body làm lộ dữ liệu nhạy cảm và cạn kiệt lưu trữ; chỉ ghi log lỗi lại mất bối cảnh khi debug. *Phòng tránh:* INFO cho sự kiện nghiệp vụ, DEBUG cho chi tiết kỹ thuật; tuyệt đối không ghi mật khẩu hay PII.
3. **Cảnh báo theo nguyên nhân kỹ thuật thay vì theo SLO** — Cảnh báo cho mọi metric kỹ thuật rời rạc (CPU, một lỗi lẻ) sinh hàng chục thông báo mỗi ngày. Hậu quả: alert fatigue — đội ngũ “miễn nhiễm” với cảnh báo, bỏ lỡ sự cố thật. *Phòng tránh:* cảnh báo dựa trên SLO/error budget gắn với trải nghiệm người dùng, phân loại mức độ nghiêm trọng rõ ràng.
4. **Cardinality explosion trong metrics** — Gắn giá trị biến thiên cao (userId, submissionId, email) làm nhãn cho Prometheus. Hậu quả: số time-series bùng nổ → quá tải hệ thống metrics, tốn chi phí, truy vấn chậm. *Phòng tránh:* chỉ dùng label cardinality thấp (status, method, service); để chi tiết cho logs/traces.
5. **Dùng tracing thay cho logging** — Tracing cho biết request đi đâu và tốn bao lâu, nhưng không giải thích *tại sao* thất bại. *Phòng tránh:* kết hợp cả hai — tracing cho cái nhìn tổng quan, logging làm rõ chi tiết.
6. **Liveness probe “đào” quá sâu** — Health check của liveness gọi tới DB hoặc downstream. Hậu quả: khi cơ sở dữ liệu xử lý chậm, Kubernetes nhận định sai rằng pod đã chết và kill tiến trình hàng loạt → sự cố lan rộng. *Phòng tránh:* liveness chỉ kiểm tra bản thân tiến trình (shallow); đưa kiểm tra dependency vào readiness.
7. **Bỏ qua observability vì “nhóm nhỏ”** — Ban đầu **docker logs** là đủ, nhưng khi hệ thống mở rộng, đọc log thủ công từ hàng chục container tiềm ẩn nhiều rủi ro vận hành. *Phòng tránh:* bắt đầu ngay với log có cấu trúc + correlation ID — bước tốn ít công sức nhất nhưng giá trị nhất.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

 **Interactive Demo**

Xem minh họa động tại companion web:

- **Truy vết phân tán (Distributed Tracing):** <distributed-tracing.html>

11.10. Tổng kết

Observability là **điều kiện tiên quyết** để vận hành microservices — không phải tính năng bổ sung. Trong monolith, một log file và một debugger đủ để tìm bug. Trong microservices, request đi qua

nhiều services, failure modes không dự đoán được — cần công cụ cho phép *hỏi câu hỏi mới* về hành vi hệ thống mà không cần deploy code mới.

Ba trụ cột — Logs, Metrics, Traces — cung cấp thông tin đa chiều: metrics phát hiện vấn đề (độ trễ tăng vọt), traces xác định vị trí (Judge Service), logs giải thích nguyên nhân (SQL timeout). Mô hình trưởng thành observability giúp nhóm xác định *mức độ cần thiết* — không phải hệ thống nào cũng cần Cấp 5. Với LMS, Cấp 3 (Proactive) là mục tiêu hợp lý.

Log có cấu trúc và Correlation ID là nền tảng — chi phí triển khai thấp nhất, tác động cao nhất. SLI/SLO/SLA cung cấp khung để chuyển từ “theo dõi metrics” sang “đo lường chất lượng dịch vụ” — đặc biệt quan trọng cho chế độ thi của LMS khi downtime ảnh hưởng trực tiếp đến trải nghiệm học tập.

Xử lý lỗi nhất quán — bộ xử lý ngoại lệ tập trung + phân loại mã lỗi — phục vụ cả client (UX tốt hơn) và đội ngũ vận hành (metrics theo nhóm mã lỗi). User tracking qua Kafka pipeline mở rộng observability từ *hệ thống* sang *hành vi học tập* — giá trị riêng mà domain giáo dục khai thác từ observability.

Phân tích KBM cho thấy observability đã có nhiều mảnh đúng hướng nhưng chưa thành một hợp đồng đầu cuối. Lộ trình chuyển đổi rõ ràng: log có cấu trúc → correlation ID → gom log tập trung → tracing → metrics + SLO. Mỗi giai đoạn mang lại giá trị ngay, không cần đợi “hoàn thiện toàn bộ” mới bắt đầu.

Ở Chương 12, chúng ta sẽ chuyển sang **Triển khai và DevOps** — containerization, Docker Compose, CI/CD pipelines, và cách deploy hệ thống microservices hoàn chỉnh.

11.11. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns* 1st Ed. — Ch.11: Developing Production-Ready Services — Health check API, Log aggregation, Distributed tracing, Application metrics
2. [4a] Sam Newman, *Building Microservices* — Ch.8: Monitoring — Logs, Metric tracking, Synthetic monitoring, Correlation IDs

Sách bổ trợ:

3. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Distributed Tracing, Domain Segregation
4. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.9: Monitoring infrastructure
5. [4b] Sam Newman, *Monolith to Microservices* — Reporting, Monitoring and Troubleshooting during migration

Nguồn trực tuyến:

- OpenTelemetry official docs — opentelemetry.io
- Micrometer docs — micrometer.io (Spring Boot observability)
- Grafana Loki — grafana.com/oss/loki (lightweight log aggregation)
- Jaeger — jaegertracing.io (distributed tracing)
- Grafana Tempo — grafana.com/oss/tempo (tracing backend trong hệ sinh thái Grafana, một lựa chọn thay cho Jaeger)
- Brendan Gregg, “USE Method” — brendangregg.com/usemethod.html

- Tom Wilkie, “RED Method” — grafana.com/blog/2018/08/02/the-red-method
- Google SRE Book, “Service Level Objectives” — sre.google/sre-book/service-level-objectives

Bài tập: Xem Phụ lục Bài tập để luyện tập:

- **11-0:** Ôn tập: ba thành phần Observability ([L1])
- **11-1:** Debugging: Bí ẩn độ trễ tăng đột biến ([L3])
- **11-2:** Observability Stack: tự xây, mua hay dùng dịch vụ managed? (Trade-off Analysis [L2])

Tài liệu tham khảo

- [2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.
- [3] Mitra, R., & Nadareishvili, I. (2020). *Microservices: Up and Running*. O'Reilly Media.
- [4a] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- [4b] Newman, S. (2019). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media.
- [5] Rocha, H. (2022). *Practical Event-Driven Microservices Architecture*. Apress.